SMART METER ENERGY ANALYTICS
USING BIG DATA

Energy Consumption Analysis, Forecasting and Anomaly Detection

Big Data Analytics

Team Members:
Poli Varshitha - CB.AI.U4AID24138
S Rajarajeswari - CB.AI.U4AID24145
Sneha A S - CB.AI.U4AID24154

Faculty: Dr.Sreeja B P

Hadoop HDFS ▪ Apache Spark ▪ Scala ▪ Spark MLlib ▪ Python ▪ Streamlit

Problem Statement, Objectives and Need for Big Data

Problem Statement

Process two smart-meter datasets of different granularity at scale: daily household readings (London) and timestamped aggregate/appliance power readings (UK-REFIT).

Project Objectives

- Analyze household energy consumption.
- Forecast future energy demand.
- Detect anomalous usage patterns.
- Compare consumption trends.
- Evaluate model performance.
- Explore results in the Streamlit dashboard.

Why Big Data?

- **Volume:** 15.7 GB logical raw data in total.
- **Velocity:** time-indexed readings.
- **Variety:** household, aggregate and appliance-level records.
- **Veracity:** missing, malformed and duplicate readings.

Observed HDFS Footprint

15.7 GB logical data
47.0 GB consumed space in HDFS (3 Datanodes used, so replication factor $= 3$)

Datasets: Source, Size, Volume, Variety and Velocity

Both the datasets used are available in Kaggle.

London Smart Meter

- Input: `daily_dataset.csv`
- Key fields: `LCLid`, `day`, `energy_sum`
- Logical HDFS size: 9.7 GB
- Dashboard scale: 5,561 households
- Household-day records: 3,510,403

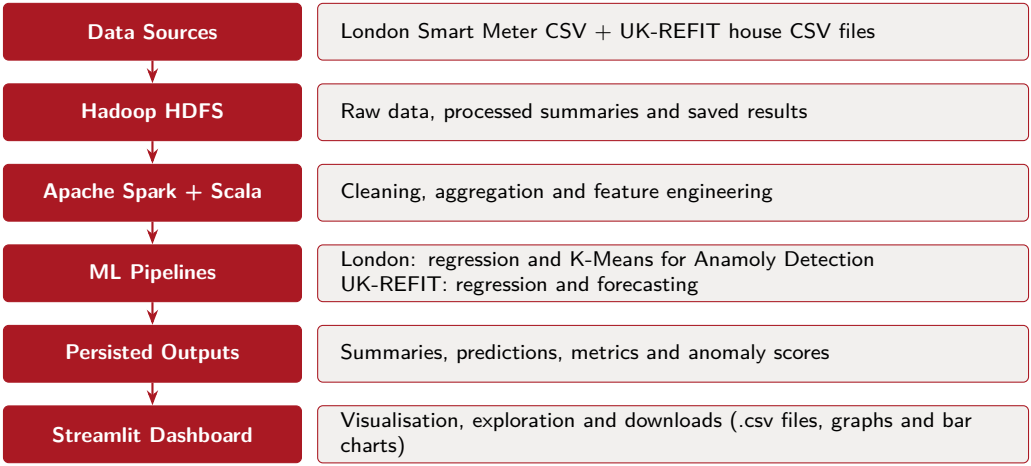
UK-REFIT

- Input pattern: `House_*.csv`
- Fields: `Time`, `Unix`, `Aggregate`
- Channels: `Appliance1`–`Appliance9`
- Logical HDFS size: 6.0 GB
- Hourly summary: 1,991,421 rows
- Daily summary: 85,797 rows

Big Data Characteristics

Volume: 15.7 GB combined
Variety: household and appliance readings
Velocity: timestamped/daily sequences
Veracity: quality checks before modelling

System Architecture



Spark Workflow: Project Pipelines and Execution

London Smart Meter

1. Read CSV from HDFS; select and cast fields.
2. Drop null/negative rows; deduplicate.
3. Group by day: total, count, average.
4. Calendar features; Random Forests, K-Means ($k = 3$).

UK-REFIT

1. Read house CSVs; derive house/date/hour.
2. Write cleaned data and summaries (Parquet).
3. Lag (1, 2, 3, 6, 24 h) and cyclical features.
4. Select by validation RMSE; test on later data.

DataFrames, DAG, Partitions and Caching

- **DataFrame:** structured transformations; used by the main workflows.
- **DAG:** Spark tracks dependencies; actions trigger execution.
- **Partitions / shuffle:** parallel work; group-by may move data.
- **Caching:** only where a reused dataset is explicitly persisted.

Scala Implementation and Project Structure

Main Scala Files

- `EnergyPreprocess.scala`
  London cleaning and aggregation
- `EnergyMLPipeline.scala`
  London regression and anomaly scoring
- `UKPreprocessing.scala`
  UK-REFIT cleaning and summaries
- `UKForecastImproved.scala`
  Forecasting and evaluation

Example: London data cleaning

```
val raw = spark.read
  .option("header", "true")
  .option("inferSchema", "true")
  .option("mode", "DROPMALFORMED")
  .csv(inputPath)

val cleaned = raw
  .select(
    col("LCLid"),
    to_date(col("day")).as("day"),
    col("energy_sum").cast("double")
      .as("energy_sum"))
  .filter(
    col("day").isNotNull &&
    col("energy_sum").isNotNull &&
    col("energy_sum") >= 0)
  .dropDuplicates("LCLid", "day")
```

Spark ML Pipelines: Features, Algorithms and Tuning

London Smart Meter

- **Aggregate:** target `avg_energy`; calendar and household-count features; Random Forest (100 trees).
- **Household:** calendar, lag-1, rolling 7-day features; Random Forest (50 trees).
- **Anomalies:** StandardScaler + K-Means ($k = 3$); 95th-percentile threshold.
- **Split:** train before 2014; test on 2014.

UK-REFIT Forecasting

- **Target:** hourly appliance mean power, `MeanW`.
- **Features:** lags 1, 2, 3, 6, 24; cyclical hour/day; IDs.
- **Candidates:** Gradient-Boosted Trees, Random Forest.
- **Baseline:** persistence using `Lag1`.
- **Split:** chronological; select by validation RMSE.

Evaluation

RMSE and MAE measure prediction error; R^2 indicates variance explained. Model choice should follow the evaluation objective.

Results: Metrics, Comparison and Business Insight

London Smart Meter

Model	RMSE	R^2
Aggregate	1.5435	−0.0048
Household	4.4994	0.8037

Targets and scales differ; do not compare directly.

UK-REFIT

Model	MAE (W)	RMSE (W)
GBT	18.330	68.642
Persistence	16.967	79.066

GBT has lower RMSE; persistence has lower MAE.

Dashboard Evidence

London: 5,561 households, 3,510,403 household-day records.
UK-REFIT: 444,294 filtered prediction pairs; filtered MAE 18.228 W, RMSE 68.630 W.

Business Insight

The London household model has a stronger R^2 than the aggregate model, though their targets differ. In UK-REFIT, model ranking depends on whether MAE or RMSE is prioritised.

Challenges, Limitations, Conclusion and Future Work

Challenges and Limitations

- Missing, duplicate readings need validation.
- Anomaly scores flag unusual patterns, not confirmed faults.
- No measured Spark speedup or live streaming is claimed.

Conclusion

HDFS stores datasets and outputs; Spark and Scala perform distributed preprocessing and ML workflows; the Streamlit dashboard exposes summaries, predictions and anomaly scores.

Future Work

- Improve missing-data handling and feature engineering.
- Tune models; compare on identical test subsets.
- Validate anomaly flags with domain experts.
- Benchmark execution time and partition strategies.

References and GitHub Repository

Technical Documentation

- Apache Hadoop: https://hadoop.apache.org/docs/
- Apache Spark 3.5.6: https://spark.apache.org/docs/3.5.6/
- Scala: https://docs.scala-lang.org/
- Streamlit: https://docs.streamlit.io/

Dataset Sources

- London Smart Meter: https://www.kaggle.com/datasets/jeanmidev/smart-meters-in-london
- UK-REFIT: https://www.kaggle.com/datasets/kyleahmurphy/uk-electrical-load

GitHub

[Insert actual repository URL]

Thank You